

Product Requirements Document (PRD)

End-to-End Fraud Detection System

Document Version: 1.0

Last Updated: November 27, 2025

Status: Active Development

Owner: ML Engineering Team

1. Executive Summary

1.1 Product Overview

The **End-to-End Fraud Detection System** is a production-grade machine learning platform designed to detect fraudulent financial transactions in real-time. The system combines advanced ML models, automated monitoring, and MLOps infrastructure to provide banks, payment processors, and fintech companies with accurate, scalable fraud prevention capabilities.

1.2 Business Objectives

- **Reduce fraud losses** by accurately identifying suspicious transactions (target: 95%+ detection rate)
- **Minimize false positives** to avoid customer friction (target: <2% false positive rate)
- **Operate at scale** with sub-100ms latency for real-time decision-making
- **Maintain model performance** through continuous monitoring and automated retraining
- **Enable rapid iteration** with MLOps infrastructure and version control
- **Support compliance** with audit trails, logging, and explainability features

1.3 Key Success Metrics

Metric	Target	Rationale
Fraud Detection Rate (Recall)	≥95%	Catch most fraud cases
False Positive Rate (1-Specificity)	<2%	Minimize customer friction
Precision	≥90%	High-confidence fraud alerts
F1-Score	≥0.92	Balance precision & recall
API Latency (p95)	<100ms	Real-time decision-making
Model Retraining Frequency	Weekly	Adapt to drift
System Uptime	99.9%	Production reliability

2. Problem Statement

2.1 The Fraud Challenge

Financial fraud causes significant losses globally:

- **Scale:** Billions of dollars lost annually across payment networks
- **Complexity:** Fraudsters continuously evolve tactics and patterns
- **Real-time pressure:** Decisions must be made in milliseconds
- **Imbalance:** Fraudulent transactions are rare (<1% of all transactions)
- **Cost of errors:**
 - False negatives: Actual fraud goes undetected (business loss)
 - False positives: Legitimate transactions blocked (customer dissatisfaction)

2.2 Current Pain Points

1. **Rule-based systems** are inflexible and miss novel fraud patterns
2. **Manual review** doesn't scale and introduces human error
3. **No adaptive learning** - systems don't improve as fraudsters evolve
4. **Lack of explainability** - regulators and customers need to understand decisions
5. **Monitoring gaps** - Model performance degradation goes undetected
6. **Deployment challenges** - Getting models to production is slow and error-prone

2.3 Proposed Solution

An ML-powered fraud detection system that:

- Uses ensemble models to capture complex fraud patterns
- Retrains automatically based on new data
- Provides real-time predictions with sub-100ms latency
- Includes comprehensive monitoring and drift detection
- Tracks model versions and enables rollback
- Offers explainability through feature importance and SHAP values
- Scales horizontally through containerization and orchestration

3. Product Scope

3.1 In Scope

- Real-time fraud prediction for individual transactions
- Model training pipeline with hyperparameter optimization
- FastAPI REST API for predictions
- Monitoring dashboard (Streamlit)
- Model versioning and registry (MLflow)
- Automated drift detection and alerts
- Containerized deployment (Docker)
- CI/CD pipeline (GitHub Actions)
- Comprehensive logging and audit trails
- Model explainability features
- Weekly automated retraining workflow

3.2 Out of Scope (Phase 1)

- Mobile application for end users
- Advanced graph-based fraud ring detection
- Blockchain integration
- Multi-language support
- Custom explainability UI (basic feature importance only)
- Real-time model updates (batch updates only)
- Geographic or device-based signals (can be added as features)

3.3 Future Roadmap (Phase 2+)

- Graph neural networks for relationship-based fraud
- Federated learning across institutions
- Advanced NLP for merchant/cardholder name matching
- Recommendation systems for legitimate transactions
- A/B testing framework for model comparisons
- Advanced explainability dashboard

4. User Personas & Use Cases

4.1 Primary Users

Persona 1: ML Engineer / Data Scientist

- **Goal:** Build, train, and monitor fraud detection models
- **Pain Points:** Model versioning, experiment tracking, deployment complexity
- **Use Cases:**
 - Train models with different architectures
 - Compare model performance across experiments
 - Deploy new models to production
 - Monitor model drift and performance
 - Debug model failures

Persona 2: Risk & Compliance Officer

- **Goal:** Understand and audit fraud detection decisions
- **Pain Points:** Lack of explainability, regulatory requirements
- **Use Cases:**
 - Review high-confidence fraud alerts
 - Generate audit reports
 - Understand model decision factors
 - Monitor false positive rates
 - Track model performance trends

Persona 3: Product Manager / Stakeholder

- **Goal:** Ensure fraud detection effectiveness and business impact
- **Pain Points:** Unclear ROI, performance visibility
- **Use Cases:**
 - Monitor fraud detection metrics
 - Track system performance
 - Understand business impact (fraud prevented, false positives)
 - Plan capacity and resources
 - Review retraining schedules

Persona 4: DevOps Engineer

- **Goal:** Deploy, scale, and maintain the system
- **Pain Points:** Deployment complexity, monitoring overhead
- **Use Cases:**
 - Deploy API to production
 - Scale infrastructure
 - Monitor system health
 - Manage containerization
 - Implement CI/CD pipeline

Persona 5: Customer Support

- **Goal:** Assist customers with blocked transactions
- **Pain Points:** Lack of clarity on why transactions were rejected
- **Use Cases:**
 - Access fraud probability for specific transactions
 - Provide customers with explanation
 - Escalate legitimate transactions for manual review
 - Track appeal process

4.2 Key Use Cases

UC1: Fraud Detection Decision

- Input: New transaction details
- Process: Pass through preprocessing, feature engineering, model inference
- Output: Fraud probability (0-1), confidence level, reasoning
- Latency: <100ms (p95)

UC2: Model Training

- Input: Historical transaction data (clean, labeled)
- Process: Train multiple models, hyperparameter tuning, evaluation
- Output: Trained model, metrics, comparison report
- Frequency: Weekly or on-demand

UC3: Model Monitoring

- Input: Live predictions and actual outcomes (labels)
- Process: Calculate drift metrics, compare distributions
- Output: Alerts if drift detected, performance reports

- Frequency: Real-time checks, daily summaries

UC4: Model Retraining

- Input: New labeled data, drift signals
- Process: Retrain model, evaluate against baseline, deploy if better
- Output: New model version, rollback capability
- Frequency: Weekly automated, or manual trigger

UC5: Dashboard Access

- Input: User login
- Process: Fetch live predictions, metrics, historical trends
- Output: Interactive dashboard with real-time data
- Users: Risk officers, ML engineers, product managers

5. Functional Requirements

5.1 Data Pipeline Requirements

FR5.1.1: Data Ingestion

- Ingest transaction data from multiple sources (batch and streaming)
- Support formats: CSV, Parquet, database connectors
- Handle data quality issues (missing values, duplicates)
- Validate schema and data types before processing

FR5.1.2: Preprocessing Pipeline

- Normalize numerical features (scaling)
- Encode categorical features (one-hot, label encoding)
- Handle missing values (imputation strategies)
- Detect and handle outliers
- Apply SMOTE for imbalanced dataset handling
- Create reproducible preprocessing workflows (versioned)

FR5.1.3: Feature Engineering

- Compute transaction velocity features (# transactions in last hour/day)
- Calculate rolling statistics (average, std, min, max over windows)
- Generate user behavior profiles
- Create amount deviation features
- Compute merchant-based features

- Support custom feature definitions

FR5.1.4: Feature Store

- Cache computed features for reuse
- Version features with metadata
- Support feature importance tracking
- Enable feature selection tools

5.2 Model Training Requirements

FR5.2.1: Model Training Pipeline

- Support training multiple algorithms:
 - Logistic Regression (baseline)
 - Decision Trees
 - Random Forest
 - XGBoost
 - Isolation Forest (unsupervised)
- Implement cross-validation (k-fold, stratified)
- Handle class imbalance with class weights and SMOTE
- Track training experiments with metadata

FR5.2.2: Hyperparameter Optimization

- Use Optuna for Bayesian optimization
- Support multiple search strategies
- Allow user-defined search spaces
- Track trials and best parameters
- Support early stopping
- Generate optimization reports

FR5.2.3: Model Evaluation

- Calculate comprehensive metrics:
 - Precision, Recall, F1-Score
 - ROC-AUC, PR-AUC
 - Confusion matrix
 - Precision-Recall curves
- Support multiple evaluation datasets (train/val/test)
- Generate comparison reports
- Flag models below performance thresholds

FR5.2.4: Model Versioning

- Save models with unique identifiers (version, timestamp)
- Store model metadata (performance, hyperparameters, training data)
- Track model lineage (parent model, changes)
- Support rollback to previous versions
- Archive old models

5.3 Prediction API Requirements

FR5.3.1: Real-time Prediction Endpoint

- Expose `/predict` endpoint accepting transaction features
- Return fraud probability (0-1) and confidence level
- Include feature importance for explainability
- Implement input validation and error handling
- Log all predictions for audit trails

FR5.3.2: Batch Prediction

- Support batch processing for historical data
- Accept multiple transactions in single request
- Return results in structured format (CSV, JSON)
- Handle large batches efficiently

FR5.3.3: API Performance

- Achieve sub-100ms latency (p95)
- Support concurrent requests (at least 100 RPS)
- Implement rate limiting and authentication
- Cache frequently requested features
- Monitor API metrics (latency, error rate)

FR5.3.4: API Versioning

- Support multiple API versions for backward compatibility
- Enable gradual rollout of new API features
- Document deprecation policies
- Maintain changelog

5.4 Monitoring & Alerting Requirements

FR5.4.1: Model Performance Monitoring

- Track real-time prediction metrics
- Monitor fraud detection rate vs. false positive rate
- Alert if metrics drop below thresholds
- Generate daily/weekly performance reports
- Support custom metric definitions

FR5.4.2: Data Drift Detection

- Monitor input feature distributions
- Detect statistical shifts (KL divergence, Kolmogorov-Smirnov test)
- Compare against baseline distributions
- Alert if drift detected
- Provide drift analysis reports

FR5.4.3: Model Drift Detection

- Track prediction distribution over time
- Detect changes in model behavior
- Compare actual vs. predicted fraud rates
- Alert if model predictions diverge from ground truth
- Generate drift investigation reports

FR5.4.4: System Health Monitoring

- Monitor API uptime and latency
- Track model inference time
- Monitor resource utilization (CPU, memory, disk)
- Alert on system failures
- Generate SLA reports

FR5.4.5: Alerting System

- Support multiple alert channels (email, Slack, PagerDuty)
- Configurable alert thresholds
- Alert severity levels (info, warning, critical)
- Alert deduplication to prevent spam
- Alert acknowledgment and resolution tracking

5.5 Dashboard Requirements (Streamlit)

FR5.5.1: Real-time Predictions Dashboard

- Display recent predictions with transaction details
- Show fraud probability distribution
- Highlight high-risk transactions
- Filter by time range, merchant, amount range
- Export prediction logs

FR5.5.2: Model Performance Dashboard

- Show confusion matrix (TP, FP, TN, FN)
- Display precision, recall, F1-score trends
- ROC curve and AUC visualization
- Precision-Recall curve
- Threshold optimization slider

FR5.5.3: Data Drift Dashboard

- Visualize feature distributions over time
- Show distribution comparisons (baseline vs. current)
- Display drift detection alerts
- Feature-by-feature drift metrics
- Recommendations for retraining

FR5.5.4: Model Drift Dashboard

- Track prediction distribution changes
- Compare fraud rate trends
- Show model performance on recent data
- Alert thresholds visualization
- Historical retraining events

FR5.5.5: System Health Dashboard

- API latency and throughput metrics
- Error rate monitoring
- Infrastructure resource usage
- Uptime percentage
- Service dependency health

5.6 MLOps & Retraining Requirements

FR5.6.1: Automated Retraining

- Schedule weekly retraining jobs
- Support manual retraining triggers
- Compare new model vs. baseline
- Auto-deploy if performance improves
- Rollback capability if issues detected

FR5.6.2: Model Registry

- Centralized model storage with metadata
- Track model versions and lineage
- Support model staging (development, staging, production)
- Enable model promotion workflows
- Archive historical models

FR5.6.3: Experiment Tracking (MLflow)

- Log all training experiments
- Track hyperparameters, metrics, artifacts
- Compare multiple experiments
- Enable reproducibility
- Generate experiment reports

FR5.6.4: CI/CD Pipeline

- Automated testing on code changes
- Code quality checks (linting, formatting)
- Model performance validation
- Docker image build and push
- Automated deployment to staging
- Manual approval for production

5.7 Explainability Requirements

FR5.7.1: Feature Importance

- Calculate feature importance for each prediction
- Show top contributing features
- Display feature contribution direction (positive/negative)
- Support global and local explanations

FR5.7.2: SHAP Values (Optional Phase 2)

- Generate SHAP values for individual predictions
- Visualize feature contributions
- Support force plots and dependency plots
- Enable detailed decision explanations

FR5.7.3: Model Interpretability

- Provide decision rules for tree-based models
- Show decision boundaries for simple models
- Generate model-specific explanations
- Support audit trail generation

6. Non-Functional Requirements

6.1 Performance Requirements

Requirement	Target	Notes
Prediction Latency (p50)	<50ms	Real-time requirement
Prediction Latency (p95)	<100ms	Industry standard
Prediction Latency (p99)	<200ms	Acceptable for monitoring
Throughput	1000+ RPS	Support peak loads
Model Training Time	<1 hour	Weekly automated retraining
Dashboard Load Time	<2 seconds	UI responsiveness
Data Processing	<30 minutes	Batch operations

6.2 Scalability Requirements

FR6.2.1: Horizontal Scalability

- Support containerized deployment (Docker, Kubernetes)
- Enable load balancing across multiple instances
- Support auto-scaling based on traffic
- Handle growing transaction volumes

FR6.2.2: Vertical Scalability

- Support larger models and datasets
- Optimize memory usage
- Support GPU acceleration for inference (optional)

- Enable distributed training (optional Phase 2)

6.3 Reliability & Availability

FR6.3.1: Uptime & SLA

- Target 99.9% uptime (43.2 minutes downtime/month)
- Support graceful degradation
- Implement circuit breakers for failures
- Enable failover mechanisms

FR6.3.2: Data Integrity

- Ensure data consistency across systems
- Implement transaction logging
- Support rollback capabilities
- Validate data at each pipeline stage

FR6.3.3: Disaster Recovery

- Maintain backup of models and data
- Support recovery from failures
- Document recovery procedures
- Test recovery mechanisms quarterly

6.4 Security Requirements

FR6.4.1: Authentication & Authorization

- Implement role-based access control (RBAC)
- Support API key authentication
- Enforce least privilege principle
- Track user actions and access logs

FR6.4.2: Data Protection

- Encrypt data at rest (AES-256)
- Encrypt data in transit (TLS/SSL)
- Implement data masking for sensitive fields
- Support PII data handling per GDPR

FR6.4.3: API Security

- Implement rate limiting
- Prevent SQL injection and XSS attacks
- Support CORS policies

- Implement request signing (optional)

FR6.4.4: Audit & Compliance

- Log all API requests and responses
- Track model changes and versions
- Generate compliance reports
- Support regulatory audits (SOC2, ISO27001)

6.5 Maintainability

FR6.5.1: Code Quality

- Maintain >80% code coverage with tests
- Use type hints (Python typing)
- Follow PEP8 style guidelines
- Implement linting and formatting checks

FR6.5.2: Documentation

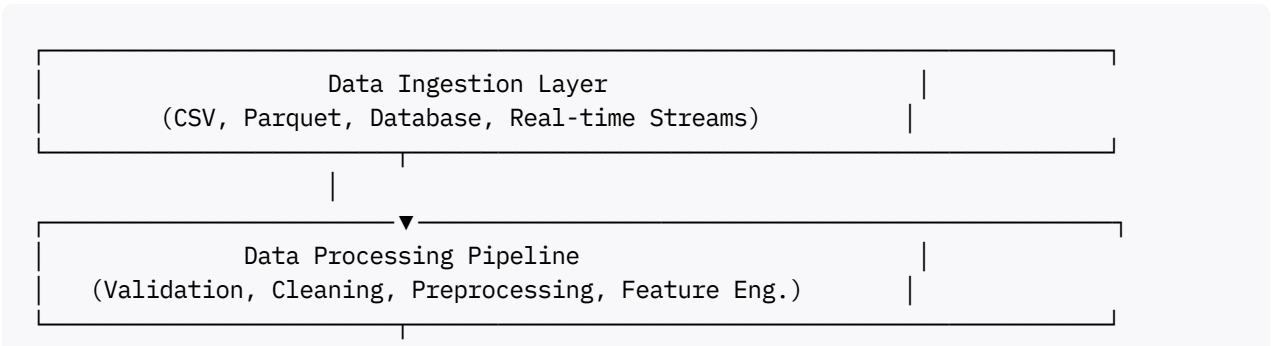
- API documentation (Swagger/OpenAPI)
- Code documentation (docstrings)
- Architecture documentation
- Deployment guides
- Troubleshooting guides

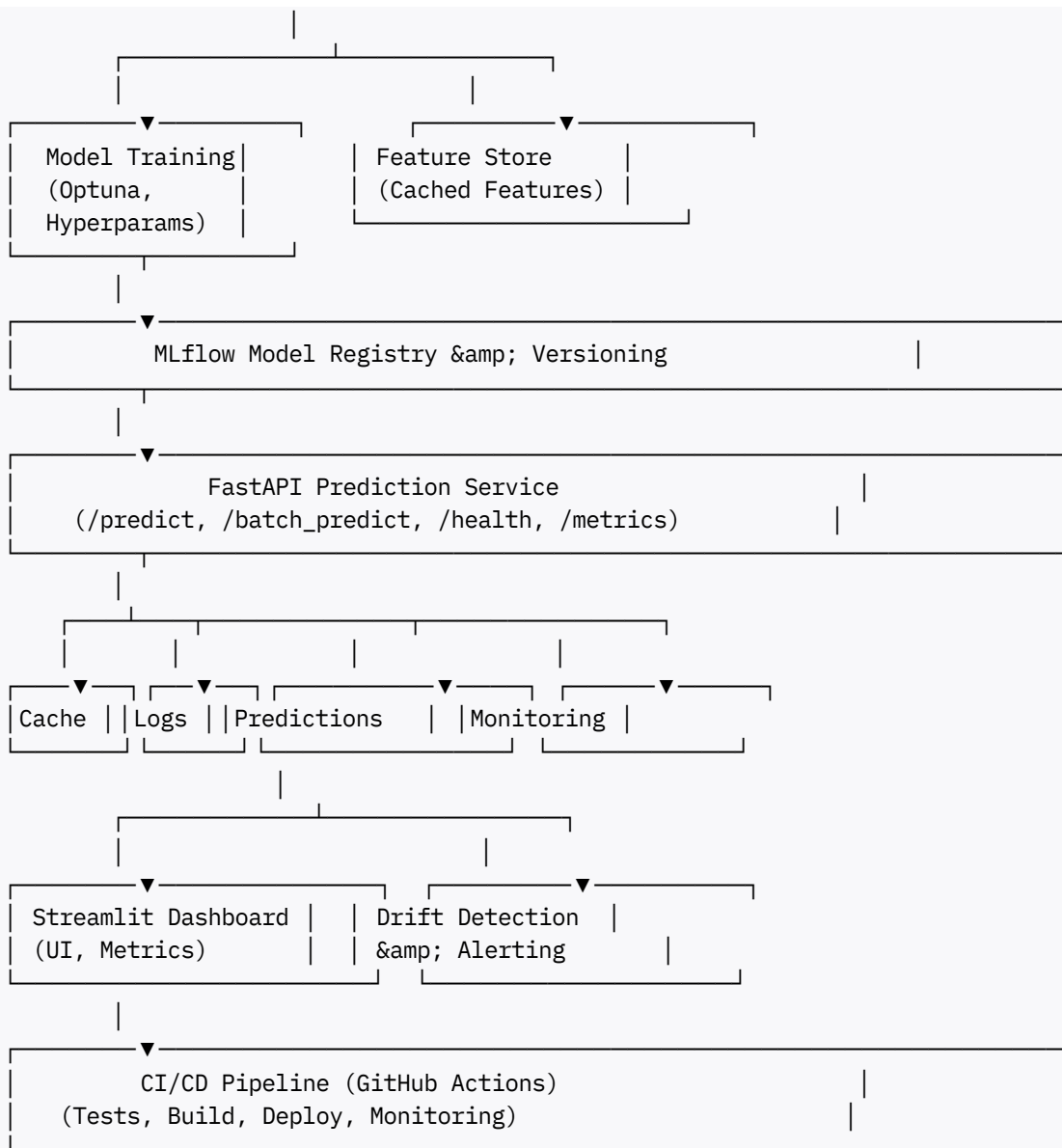
FR6.5.3: Logging & Observability

- Structured logging (JSON format)
- Log levels: DEBUG, INFO, WARNING, ERROR, CRITICAL
- Centralized log aggregation
- Support tracing across services

7. Technical Architecture

7.1 System Components





7.2 Technology Stack

Layer	Technology	Rationale
Data Processing	Pandas, NumPy	Industry standard for ML data handling
ML Framework	Scikit-learn, XGBoost, LightGBM	Mature, production-ready
Imbalance	Imbalanced-learn (SMOTE)	Handle fraud imbalance
Hyperparameter Tuning	Optuna	Fast, efficient Bayesian optimization
Model Registry	MLflow	Industry standard for model tracking
API Framework	FastAPI	Fast, async, modern Python web framework
Monitoring UI	Streamlit	Rapid dashboard development
Containerization	Docker	Reproducible deployments
Orchestration	Docker Compose / Kubernetes	Scalable deployment

Layer	Technology	Rationale
CI/CD	GitHub Actions	Integrated with GitHub, easy setup
Logging	Python logging + ELK	Structured logging and analysis
Testing	pytest, pytest-cov	Comprehensive test coverage
Data Versioning	DVC (optional)	Track data and model versioning

8. Data Requirements

8.1 Input Data Specifications

Dataset: Fraud detection datasets (e.g., Kaggle Credit Card Fraud Dataset)

Transaction Record Format:

```
{
  "transaction_id": "unique_id",
  "timestamp": "2025-01-15T10:30:45Z",
  "amount": 150.50,
  "merchant_id": "merchant_xyz",
  "merchant_category": "grocery",
  "cardholder_id": "user_123",
  "is_fraud": 1,
  "v1": -1.3598071336738206,
  "v2": -0.0747971826222904,
  "...": "28 anonymized features V1-V28",
  "time": 0
}
```

8.2 Data Volume

Metric	Target	Notes
Training Dataset	200k-500k transactions	Sufficient for model training
Test Dataset	50k-100k transactions	Holdout evaluation
Daily Ingestion	1M+ transactions	Real-world production volume
Historical Archive	2+ years	Long-term trending

8.3 Data Quality Standards

- Missing value rate: <5% per feature
- Duplicate rate: <1%
- Invalid records: <0.1%
- Outliers: Handled via domain knowledge

- Label quality: >95% accuracy (human-verified labels)

8.4 Feature Set

Provided Features (28 anonymized + metadata):

- V1-V28: PCA-transformed features from original transaction data
- Time: Seconds elapsed between first transaction and current
- Amount: Transaction amount

Engineered Features:

- Transaction velocity (transactions per hour/day by user)
- Merchant frequency (transaction count by merchant)
- Amount deviation (deviation from user average)
- Time-of-day features
- Device fingerprint features
- Geographic features

9. Success Criteria & KPIs

9.1 Model Performance KPIs

KPI	Target	Current	Owner
Fraud Detection Rate (Recall)	≥95%	TBD	Data Science
False Positive Rate	<2%	TBD	Data Science
Precision	≥90%	TBD	Data Science
F1-Score	≥0.92	TBD	Data Science
ROC-AUC	≥0.98	TBD	Data Science
PR-AUC	≥0.95	TBD	Data Science

9.2 System Performance KPIs

KPI	Target	Current	Owner
API Latency (p95)	<100ms	TBD	DevOps
API Throughput	1000+ RPS	TBD	DevOps
System Uptime	99.9%	TBD	DevOps
Error Rate	<0.1%	TBD	DevOps
Model Training Time	<1 hour	TBD	ML Eng

9.3 Business KPIs

KPI	Target	Current	Owner
Fraud Cases Caught	95% of known fraud	TBD	Risk
False Positive Cost	<\$50k/month	TBD	Risk
Model Improvement	10% YoY	TBD	PM
Time-to-Market	8 weeks	TBD	PM

9.4 Success Milestones

Milestone	Target Date	Status
Data pipeline complete	Week 2	
Model training working	Week 3	
1st baseline model (>90% recall)	Week 4	
API deployment	Week 5	
Monitoring dashboard live	Week 6	
Automated retraining	Week 7	
MLOps pipeline complete	Week 8	
Production deployment	Week 9	

10. Constraints & Assumptions

10.1 Constraints

Technical:

- Python 3.9+ required
- Must support cloud deployment (AWS, GCP, Azure)
- API must handle 1000+ concurrent requests
- Model inference must complete in <100ms

Business:

- Budget limited to open-source and low-cost tools
- Timeline: 8-12 weeks for MVP
- Team size: 2-3 ML engineers, 1 DevOps

Regulatory:

- GDPR compliance for EU transactions
- PCI-DSS compliance for payment data

- SOC2 audit requirements

10.2 Assumptions

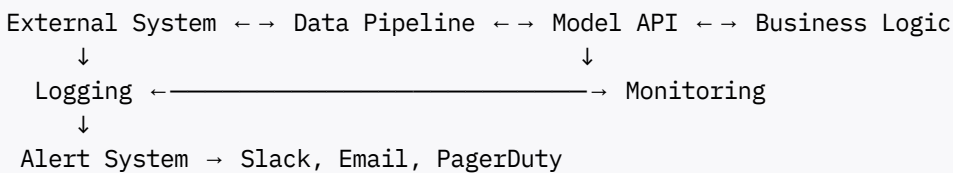
- Historical labeled fraud data is available
- Transaction features are reliable and consistent
- Fraud patterns are learnable from historical data
- Ground truth labels become available within 7 days
- Infrastructure (cloud, servers) is provided by DevOps
- Domain experts can define initial feature set

11. Dependencies & Integration

11.1 External Dependencies

- **Data Sources:** Transaction database, payment gateway APIs
- **Labeling:** Manual fraud confirmation process
- **Infrastructure:** Cloud provider (AWS/GCP), Kubernetes cluster
- **Monitoring:** Log aggregation service (Splunk, ELK)

11.2 Integration Points



12. Release Plan & Timeline

12.1 Phase 1: MVP (Weeks 1-8)

Week 1-2: Foundation

- [] Set up project structure
- [] Data pipeline (ingestion, preprocessing)
- [] EDA and feature engineering

Week 3-4: Model Development

- [] Train baseline models
- [] Hyperparameter tuning (Optuna)

- ☐ Model evaluation and comparison

Week 5: Deployment

- ☐ FastAPI API development
- ☐ Model versioning with MLflow
- ☐ Docker containerization

Week 6: Monitoring

- ☐ Streamlit dashboard
- ☐ Drift detection implementation
- ☐ Alerting system

Week 7-8: MLOps & CI/CD

- ☐ Automated retraining pipeline
- ☐ GitHub Actions CI/CD
- ☐ Production deployment and testing

12.2 Phase 2: Enhancement (Weeks 9-12)

- Advanced explainability (SHAP values)
- Graph-based fraud detection
- A/B testing framework
- Enhanced monitoring features

12.3 Phase 3: Scale (Months 4-6)

- Distributed training
- Real-time model updates
- Advanced NLP features
- Multi-region deployment

13. Risk Management

13.1 Identified Risks

Risk	Probability	Impact	Mitigation
Data quality issues	High	High	Implement data validation, quality checks
Model performance degradation	Medium	High	Drift detection, automated retraining
API latency failures	Medium	High	Performance testing, load testing, caching
Model bias	Medium	Medium	Fairness audits, bias detection

Risk	Probability	Impact	Mitigation
Security vulnerabilities	Low	Critical	Security reviews, penetration testing
Resource constraints	Medium	Medium	Auto-scaling, load balancing

13.2 Risk Mitigation Strategies

1. **Data Quality:** Automated data validation at ingestion
2. **Model Drift:** Weekly drift detection and alerts
3. **Performance:** Load testing before deployment
4. **Security:** Regular security audits and penetration testing
5. **Scalability:** Infrastructure auto-scaling setup

14. Glossary & Definitions

Term	Definition
Fraud Detection Rate	Percentage of actual fraud cases correctly identified (Recall)
False Positive Rate	Percentage of legitimate transactions incorrectly flagged as fraud
ROC-AUC	Area Under the Receiver Operating Characteristic curve (model discrimination)
Data Drift	Change in input feature distributions over time
Concept Drift	Change in fraud patterns or model behavior over time
Hyperparameter	Parameters set before training (learning rate, tree depth)
SMOTE	Synthetic Minority Over-sampling Technique (handle imbalance)
MLOps	Machine Learning Operations (model deployment, monitoring, retraining)
Precision	Of predicted frauds, how many are actually fraud
Recall	Of actual frauds, how many were detected
F1-Score	Harmonic mean of precision and recall
Feature Importance	Contribution of each feature to model prediction
Model Versioning	Tracking different versions of deployed models

15. Appendix: Additional Considerations

15.1 Compliance & Regulatory

- **GDPR:** Ensure data privacy and user consent
- **PCI-DSS:** Payment data must be encrypted and secure
- **SOC2:** Audit trails and access controls required

- **FAIR:** Fair Lending practices (no discrimination)

15.2 Future Enhancements

1. **Real-time Learning:** Update models as new labels arrive
2. **Ensemble Methods:** Combine multiple models for better accuracy
3. **Graph Analysis:** Detect fraud rings and relationships
4. **Federated Learning:** Train models across multiple institutions
5. **Causality:** Understand causal relationships in fraud

15.3 References & Resources

- [Fraud Detection Best Practices](#)
- [MLflow Documentation](#)
- [FastAPI Guide](#)
- [Scikit-learn Documentation](#)
- [NIST ML Standards](#)

16. Approval & Sign-Off

Role	Name	Date	Signature
Product Manager	TBD		
Engineering Lead	TBD		
Data Science Lead	TBD		
DevOps Lead	TBD		
Compliance Officer	TBD		

Document Control:

- **Version:** 1.0
- **Last Updated:** November 27, 2025
- **Next Review:** December 27, 2025
- **Owner:** ML Engineering Team
- **Repository:** GitHub (link TBD)